

Lecture #3

Instructor: Yunseok Choi
(ys.choi@skku.edu)

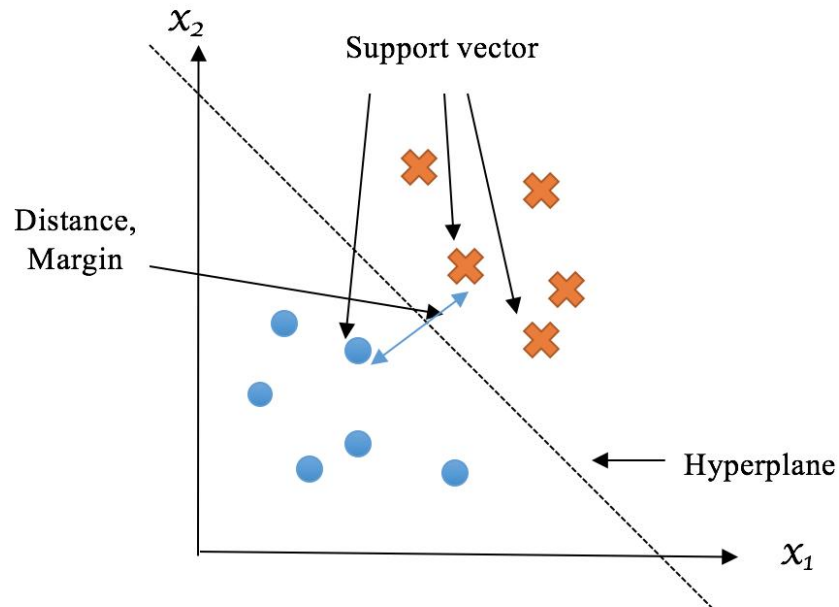
Support Vector Machine

SVM Support Vector Machine

- 클래스 별 데이터를 가장 잘 구분하는
최적의 **하이퍼플레인 (Hyperplane)**을 찾는 알고리즘
- 하이퍼플레인 : 데이터가 분포되어 있는 n 차원 피쳐 공간을
두 공간으로 나누는 $n - 1$ 차원의 평면
 - 예: 2차원 피쳐 공간에서 하이퍼플레인은 '선' (line),
3차원 피쳐 공간에서 하이퍼플레인은 '평면' (surface)

최적화 된 하이퍼플레인

- 하이퍼플레인에 의해 양분된 두 공간 내 있는 가장 가까운 데이터 포인트(벡터) 간의 거리(Margin)가 최대인 상
E → 가장 가까운 데이터 포인트: "서포트 벡터 (Support Vector)"



단계 1: 하이퍼플레인 표현하기

- 하이퍼플레인도 부분공간으로 생각하면 된다. (벡터들의 집합)
- 2차원 공간의 하이퍼플레인을 생각했을 때, 하이퍼플레인이 포함하는 벡터(데이터 포인트)들은 다음식으로 정의될 수 있다.

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

흔히 선을 표현할 때 사용하는
일차식 $ax + by + c = 0$ 을
벡터로 표현했다고 생각하면 된다.

단계 1: 하이퍼플레인 표현하기

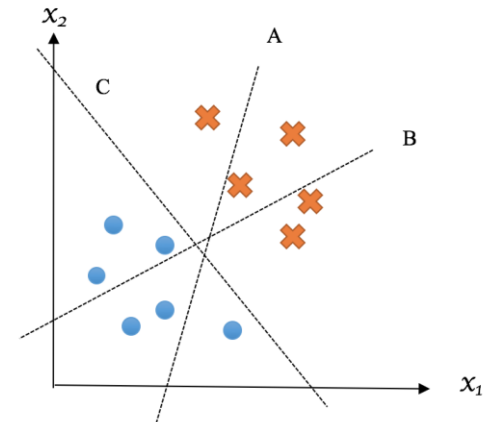
"최적의 하이퍼플레인 조건1"

- 두 클래스의 데이터들을 양분하기 위해서는 하이퍼플레인
아래 조건을 만족해야 한다.
 - 2개 클래스 중 한 쪽에 속한 모든 데이터 x 에 대해 $w \cdot x + b > 0$
 - 2개 클래스 중 다른 쪽에 속한 모든 데이터 x 에 대해 $w \cdot x + b < 0$

단계 1: 하이퍼플레인 표현하기

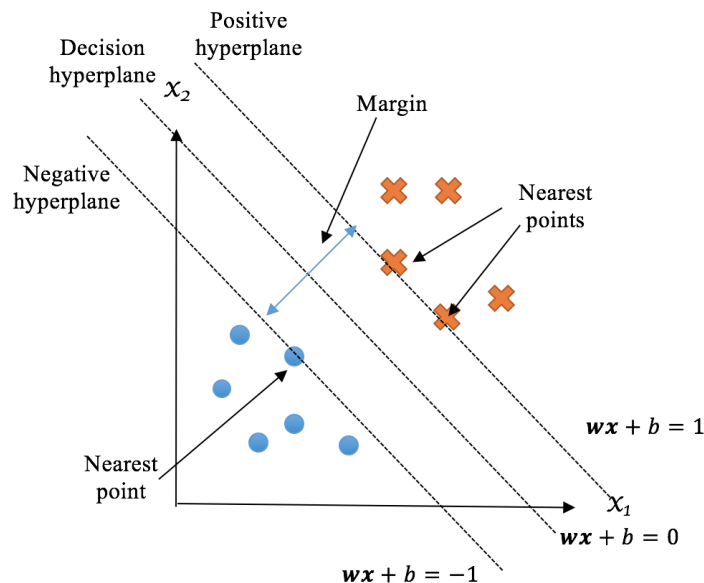
- 두 클래스의 데이터들을 양분하기 위해서는 하이퍼플레인 은 아래 조건을 만족해야 한다.
 - 2개 클래스 중 한 쪽에 속한 모든 데이터 x 에 대해 $w \cdot x + b > 0$
 - 2개 클래스 중 다른 쪽에 속한 모든 데이터 x 에 대해 $w \cdot x + b < 0$
- 하지만, 이 조건을 만족하는 하이퍼플레인 은 무수히 많을 수 있다.

➔ 조건이 더 필요!



단계 2: 서포트 벡터 표현하기

- 두 공간 내 가장 가까운 벡터들 사이의 거리가 멀면 멀 수록 두 공간을 잘 가르는 하이퍼플레인일 것
(즉 Margin이 크면 클 수록 좋은 하이퍼플레인)
➔ 이를 수학적으로 표현해 보자.



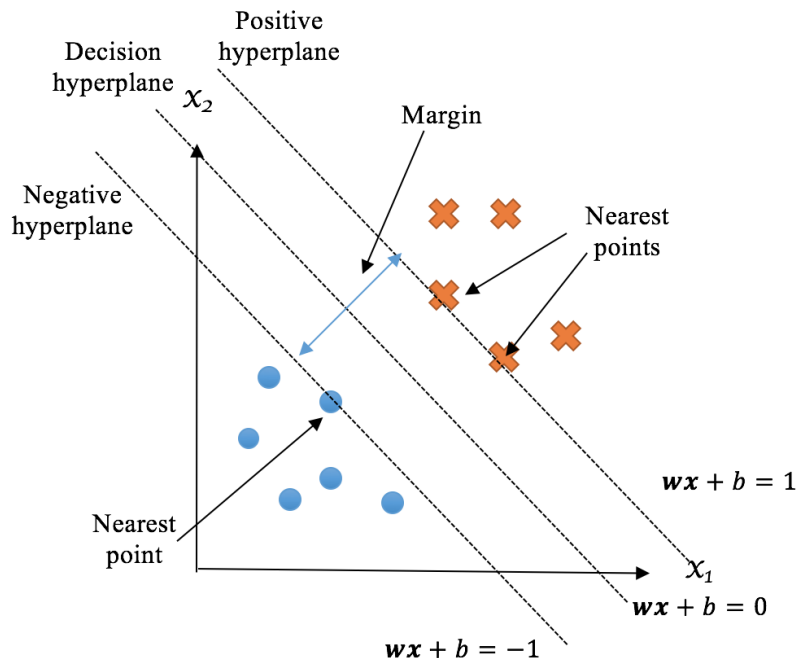
단계 2: 서포트 벡터 표현하기

- 디시전 하이퍼플레인 : 두 공간을 양분하는 하이퍼플레인 (Decision Hyperplane)
- 양성 하이퍼플레인 (Positive Hyperplane)
 - 디시전 하이퍼플레인보다 큰 곳에서 평행하는 하이퍼플레인
 - 위치한 공간에서 디시전 하이퍼플레인보다 가장 가까운 데이터를 포함
- 음성 하이퍼플레인 (Negative Hyperplane)
 - 디시전 하이퍼플레인보다 작은 곳에서 평행하는 하이퍼플레인
 - 위치한 공간에서 디시전 하이퍼플레인보다 가장 가까운 데이터를 포함

양성/음성 하이퍼플레인 내 데이터가 서포트 벡터가 된다.

단계 2: 서포트 벡터 표현하기

- 디시전 하이퍼플레인: $\mathbf{w} \cdot \mathbf{x} + b = 0$
- 양성 하이퍼플레인: $\mathbf{w} \cdot \mathbf{x}^{(p)} + b = 1$
- 음성 하이퍼플레인: $\mathbf{w} \cdot \mathbf{x}^{(n)} + b = -1$



1과 -1은 계산 편의상
합이 0이 되게 하기 위해 사용
(b 로 조정 가능하기 때문에
사실 어떤 수여도 상관없음)

단계 3: Margin 표현하기

- 직선과 점이 벡터로 표현되었을 뿐 똑같은 식을 사용하면 된다.
- 양성 하이퍼플레인의 점 $\mathbf{x}^{(p)}$ 에서 직선 $\mathbf{w} \cdot \mathbf{x} + b = 0$ 사이의 거리 $d^{(p)}$

$$d^{(p)} = \frac{|\mathbf{w} \cdot \mathbf{x}^{(p)} + b|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|} \quad (\mathbf{w} \cdot \mathbf{x}^{(p)} + b = 1)$$

- 음성 하이퍼플레인의 점 $\mathbf{x}^{(n)}$ 에서 직선 $\mathbf{w} \cdot \mathbf{x} + b = 0$ 사이의 거리 $d^{(n)}$

$$d^{(n)} = \frac{|\mathbf{w} \cdot \mathbf{x}^{(n)} + b|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|} \quad (\mathbf{w} \cdot \mathbf{x}^{(n)} + b = -1)$$

단계 3: Margin 표현하기

- 따라서 Margin은 아래와 같이 표현될 수 있다.

$$d^{(p)} + d^{(n)} = \frac{2}{\|\mathbf{w}\|}$$

“최적의 하이퍼플레인 조건 2”

= 서포트 벡터 사이 거리가 최대가 되도록 하는

$\|\mathbf{w}\|$ 가 최소인 하이퍼플레인

단계 4: 최적의 하이퍼플레인 찾기

<조건 1> - 데이터 양분

- 2개 클래스 중 한 쪽에 속한 모든 데이터 x 에 대해 $w \cdot x + b > 0$
- 2개 클래스 중 다른 쪽에 속한 모든 데이터 x 에 대해 $w \cdot x + b < 0$



이를 양성/음성 하이퍼플레인 표현으로 다시 바꿔보면

$$w \cdot x^{(i)} + b \geq 1 \text{ if } y^{(i)} = 1$$

$$w \cdot x^{(i)} + b \leq -1 \text{ if } y^{(i)} = -1$$

$(x^{(i)}, y^{(i)})$ 는 i 번째 데이터의 피처와 클래스 레이블을 표현



이 두 식을 하나로 정리하면

$$y^{(i)}(w \cdot x^{(i)} + b) \geq 1$$

단계 4: 최적의 하이퍼플레인 찾기

<조건 1> - 데이터 양분

- 모든 학습 데이터, $(\mathbf{x}^{(i)}, y^{(i)})$ 에 대하여 ($i = 1, 2, 3, \dots m$)
 $y^{(i)}(\mathbf{w} \cdot \mathbf{x}^{(i)} + b) \geq 1$ 을 만족

<조건 2> - 최대 Margin

- $\frac{\|\mathbf{w}\|}{2}$ 이 최소

이 조건들을 만족하는 하이퍼플레인을
찾는 것이 SVM의 학습 과정

단계 4: 최적의 하이퍼플레인 찾기 (추가)

<조건 1> - 데이터 양분

- 모든 학습 데이터, $(\mathbf{x}^{(i)}, y^{(i)})$ 에 대하여 ($i = 1, 2, 3, \dots m$)

$y^{(i)}(\mathbf{w} \cdot \mathbf{x}^{(i)} + b) \geq 1$ 을 만족

<조건 2> - 최대 Margin

- $\frac{\|\mathbf{w}\|}{2}$ 이 최소

하지만! 현실적으로 많은 경우에

조건 1에 대해서 모든 학습 데이터를 모두 다 양분하는
'선형 식'을 찾기는 어렵다. → 완화된 조건으로 수정이 필요

단계 4: 최적의 하이퍼플레인 찾기 (추가)

학습 단계에서 현재 선형식으로 학습 데이터를 분류해 봤을 때,
올바르게 분류(양분)되지 못한 경우를 다음과 같이 수치화 한다.

$$\zeta^{(i)} = \begin{cases} 1 - y^{(i)}(\mathbf{w}\mathbf{x}^{(i)} + b), & \text{if misclassified} \\ 0, & \text{otherwise} \end{cases}$$

$$Loss = \frac{\sum_{i=1}^m \zeta^{(i)}}{m}$$

이를 Error라 하기도 하고, **힌지 손실(Hinge Loss)**라고 하기도 한다.

Error가 0이면 이상적이겠지만, 실질적으로 존재할 수 있기 때문에

학습 과정에서 Error가 최소가 되게 하는 조건을 추가해 주면 좋다.

단계 4: 최적의 하이퍼플레인 찾기

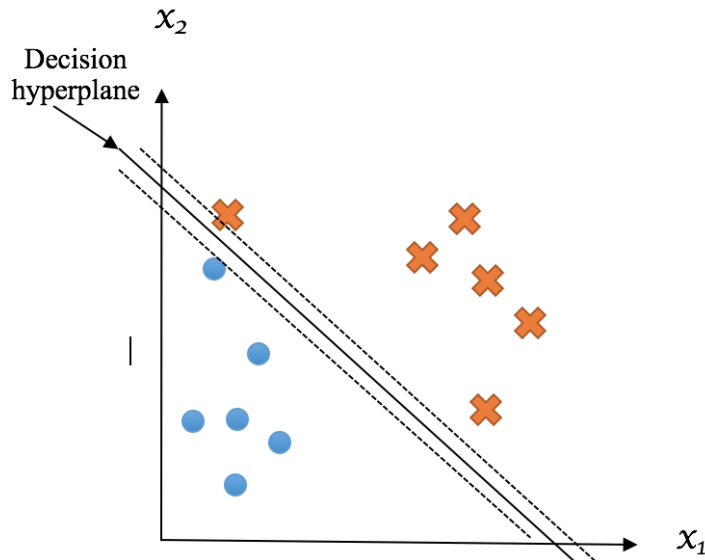
<조건> - 완화된 데이터 양분 + 최대 Margin

■ $\frac{\|w\|}{2} + C \frac{\sum_{i=1}^m \zeta^{(i)}}{m}$ 이 최소

* C 는 Margin과 Error 중 어떤 것을 더 중요하게 볼 것인지 결정하는 하이퍼 파라미터

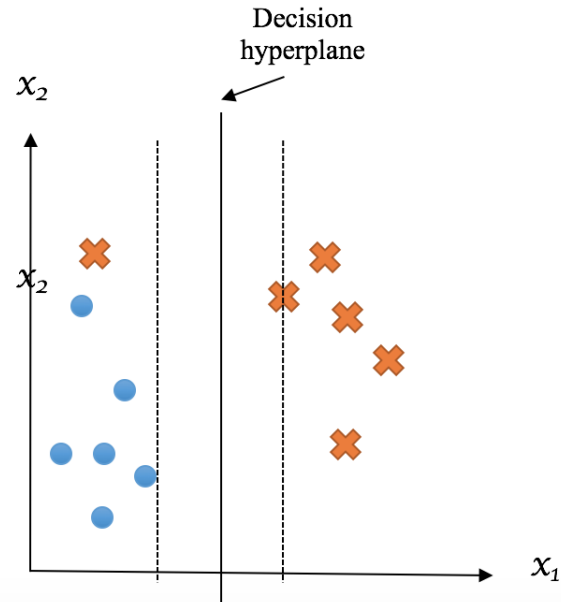
- C 가 크면 $\rightarrow$ Error 값을 더 고려 $\rightarrow$ 학습 데이터에 오버피팅
- C 가 작으면 $\rightarrow$ Error 값을 거의 고려 안함 $\rightarrow$ 학습 데이터에 대한 정확도 떨어질 수 있음

단계 4: 최적의 하이퍼플레인 찾기



c 가 큰 경우

Margin은 작고 미분류 에러 X



c 가 작은 경우

Margin은 크나 미분류 에러 존재

c 로 모델의 바이어스와 분산 사이 밸런스를 결정한다고 볼 수 있다.

➔ 이러한 하이퍼 파라미터를 찾는 과정을 '모델 튜닝'이라고 하며

다른 파라미터 상황에서 모델의 성능(예: 교차검증 결과)으로 최적의 파라미터를 결정

단계 5: 학습 후 예측

학습을 통해 최적의 w 와 b 를 찾아 다음 식을 완성했다면

$$w \cdot x + b$$

새로운 x' 가 주어졌을 때 x' 의 레이블 y' 는 다음과 같이 결정

$$y' = \begin{cases} 1, & \text{if } \mathbf{w}\mathbf{x}' + b > 0 \\ -1, & \text{if } \mathbf{w}\mathbf{x}' + b < 0 \end{cases}$$

SVM– Binary Classification

문제 : 토픽 분류

- Newsgroup20 데이터에 있는 포스트들을 주제별로 분류하는 SVM을 만들어 보자.
- 먼저, 문제를 간단하게 하기 위해 comp.graphics와 sci.space 두 개 주제로 한정해서 이진 분류 문제로 풀어보자.

1. 데이터 불러오기

```
from sklearn.datasets import fetch_20newsgroups

categories = ['comp.graphics', 'sci.space']

data_train = fetch_20newsgroups(subset='train',
                                categories=categories, random_state=42)

data_test = fetch_20newsgroups(subset='test',
                                categories=categories, random_state=42)
```

2. 데이터 전처리

```
from nltk.corpus import names
from nltk.stem import WordNetLemmatizer
```

```
all_names = set(names.words())
lemmatizer = WordNetLemmatizer()
```

앞서 사용했던 전처리기

```
def clean_text(docs):
    cleaned_docs = []
    for doc in docs:
        lemmatized_list = [ lemmatizer.lemmatize( word.lower() )
                             for word in doc.split()
                             if word.isalpha() and word not in all_names ]
        cleaned_docs.append(' '.join(lemmatized_list))

    return cleaned_docs
```

```
cleaned_train = clean_text(data_train.data)
label_train = data_train.target
```

```
cleaned_test = clean_text(data_test.data)
label_test = data_test.target
```

```
len(label_train), len(label_test)
```

(1177, 783)

3. 피처 추출 (TF-IDF) → SVM 학습

피처 추출

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(stop_words='english', max_features=8000)

term_docs_train = tfidf_vectorizer.fit_transform(cleaned_train)
term_docs_test = tfidf_vectorizer.transform(cleaned_test)
```

SVM 학습

Kernel에 대해서는 다음에 설명 ☺

```
from sklearn.svm import SVC

svm = SVC(kernel='linear', C=1.0, random_state=42)

svm.fit(term_docs_train, label_train)
```

```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
    decision_function_shape='ovr', degree=3, gamma='auto', kernel='linear',
    max_iter=-1, probability=False, random_state=42, shrinking=True,
    tol=0.001, verbose=False)
```


4. 성능 평가

일단 간단히 정확도만 보면 ...

```
accuracy = svm.score(term_docs_test, label_test)
print('The accuracy on testing set is: {0:.1f}%'.format(accuracy*100))
```

SVM – Multiclass Classification

다중 클래스 분류 문제

- 앞서 봤던 SVM 모델은 데이터를 “양분”하는 하이퍼플레인을 찾기 때문에 **이진 분류 문제**에 대한 모델이다.
- 만약 클래스가 세 개 이상인 **다중 클래스 분류** 문제에 대해서는 SVM을 사용할 수 있을까?

다중 클래스 분류 문제

- 앞서 봤던 SVM 모델은 데이터를 “양분”하는 하이퍼플레인을 찾기 때문에 **이진 분류 문제**에 대한 모델이다.
- 만약 클래스가 세 개 이상인 **다중 클래스 분류** 문제에 대해서는 SVM을 사용할 수 있을까?

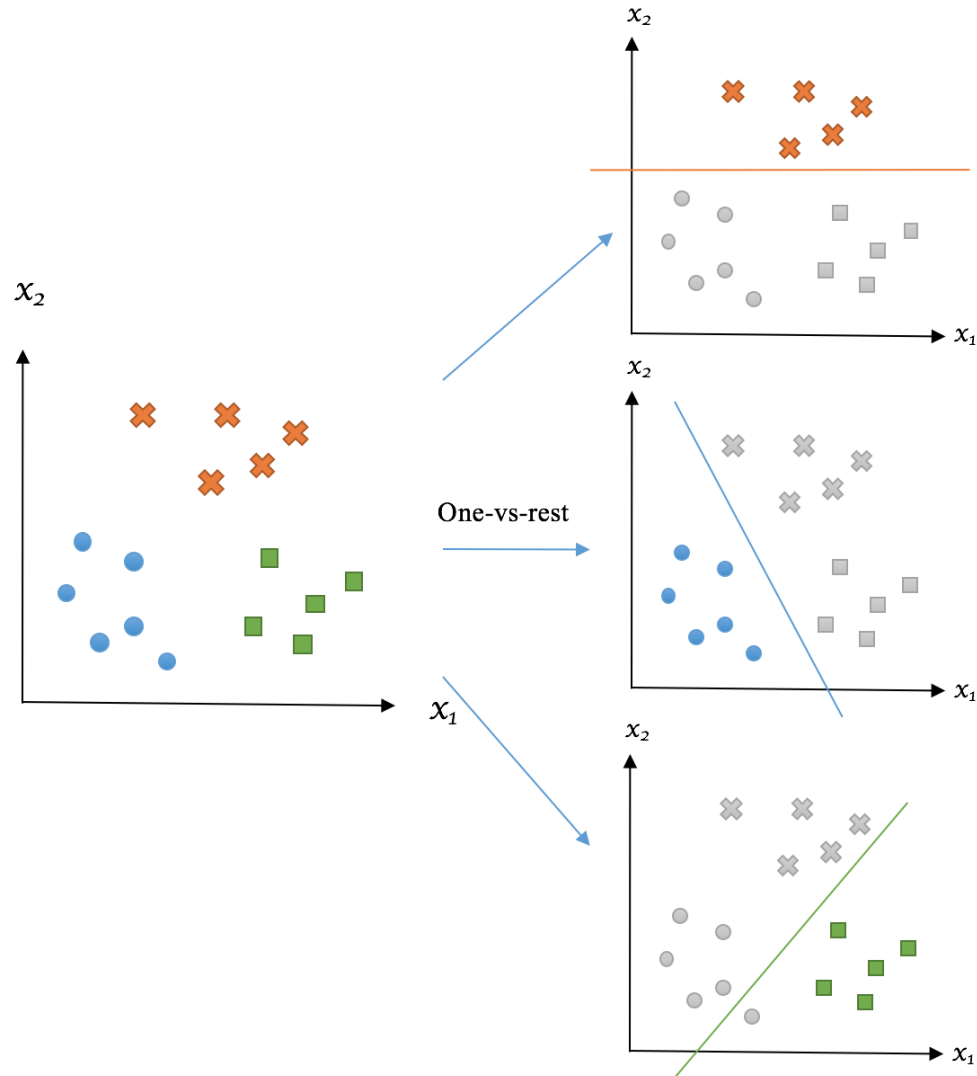
사용할 수 있다!

각 클래스 별로 “**해당 클래스**”와 “**그 외 다른 클래스들**”을 가르는 하이퍼 플레인을 찾는다고 생각하면 된다.

다중 클래스 SVM : (1) 일대다 방법

- K -클래스 문제에 대해 K 개의 (서로 다른) 이진 SVM분류기를 만들어낸다.
 - k 번째 분류기의 경우 k 번째 클래스를 양성 케이스로 처리하고 나머지 $K - 1$ 개의 클래스는 음성 케이스로 처리한다.
 - ➔ 이를 학습하여 하이퍼플레인 $\mathbf{w}_k \mathbf{x} + b_k = 0$ 을 찾는다.
- 학습 후 $\mathbf{x}'$ 의 클래스 y' 는 이진 분류기의 $\mathbf{w}_k \mathbf{x} + b_k$ 의 값이 가장 큰 값을 가지는 클래스 k 를 $\mathbf{x}'$ 의 클래스로 결정한다.
$$y' = \operatorname{argmax}_{k=1,\dots,K} (\mathbf{w}_k \mathbf{x} + b_k)$$

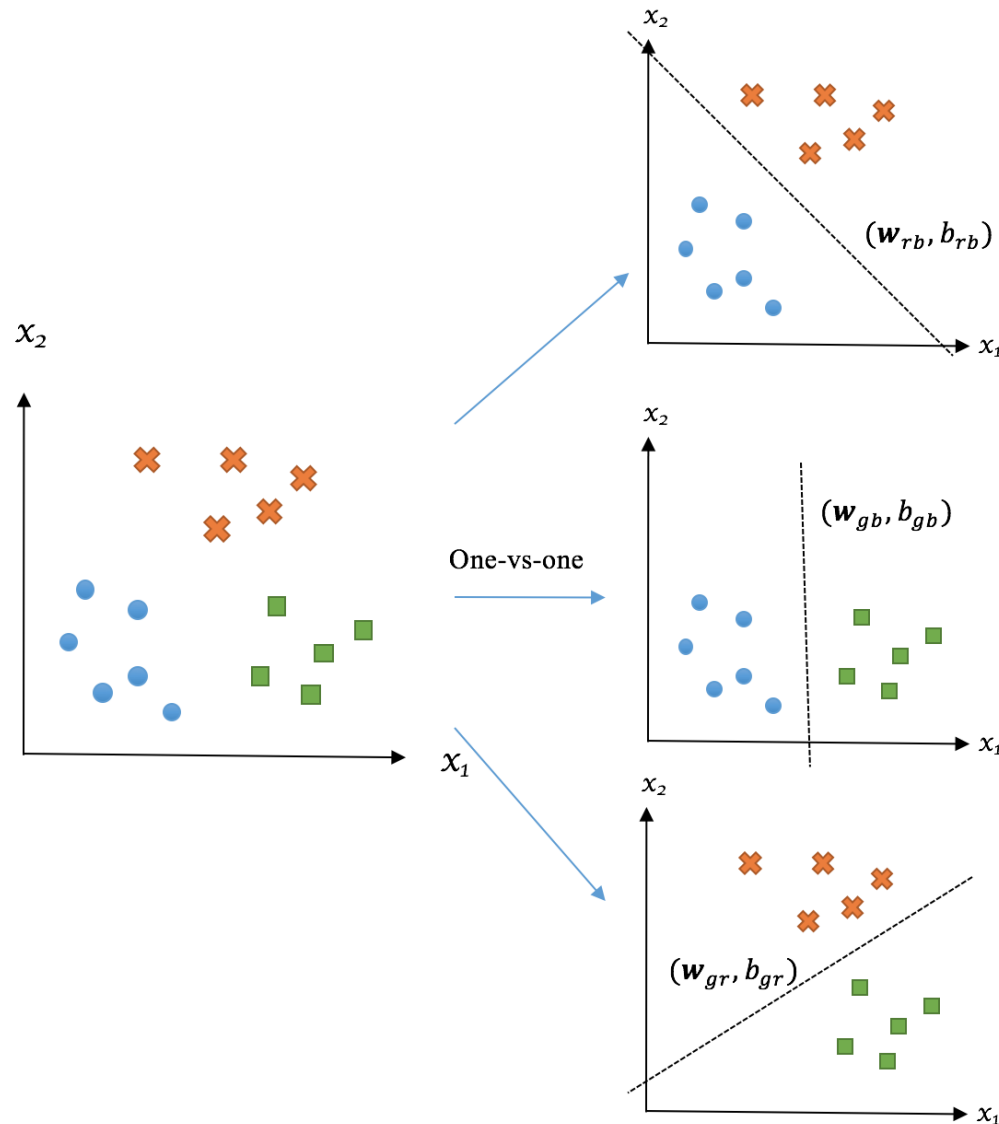
다중 클래스 SVM : (1) 일대다 방법



다중 클래스 SVM : (2) 일대일 방법

- 클래스의 각 쌍을 구분하는 이진 SVM 분류기를 생성
 - 쌍 비교 방법 (Pairwise comparison)
 - K 개 클래스가 있으면 $\frac{K(K-1)}{2}$ 개 분류기 생성
- 학습 후, 새로운 데이터 $\mathbf{x}'$ 를 분류할 때는 모든 이진 분류기에
데이터를 넣고 결과를 뽑아보고 가장 많이 예측된 클래스를
 $\mathbf{x}'$ 의 최종 예측 클래스 $\mathbf{y}'$ 으로 결정한다.

다중 클래스 SVM : (2) 일대일 방법



일대다 vs. 일대일

- 일대일 방법은 일대다 방법보다 분류기가 훨씬 더 많이 만들어진다.
 - 일대일 : $K(K-1)/2$
 - 일대다 : K
- 일대다 방법은 한 분류기를 학습시킬 때 학습데이터로 전체 데이터를 사용하지만, 일대일 방법은 대상이 되는 두 클래스에 속하는 데이터만 사용한다.
 - 한 분류기를 학습하는 상황에서는 일대일이 메모리 활용, 컴퓨팅 계산 효율이 더 좋다.

다중 클래스 SVM in Python

- 다중 클래스 데이터 준비 (5개 클래스)

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer

categories = [
    'alt.atheism', 'talk.religion.misc', 'comp.graphics',
    'sci.space', 'rec.sport.hockey']

data_train = fetch_20newsgroups(subset='train', categories=categories, random_state=42)
data_test = fetch_20newsgroups(subset='test', categories=categories, random_state=42)

cleaned_train = clean_text(data_train.data)
label_train = data_train.target

cleaned_test = clean_text(data_test.data)
label_test = data_test.target

tfidf_vectorizer = TfidfVectorizer(stop_words='english', max_features=8000)
term_docs_train = tfidf_vectorizer.fit_transform(cleaned_train)
term_docs_test = tfidf_vectorizer.transform(cleaned_test)
```

다중 클래스 SVM in Python

- 다중 클래스 SVM 학습 및 정확도 출력

```
from sklearn.svm import SVC

svm = SVC(kernel='linear', C=1.0, random_state=42)

svm.fit(term_docs_train, label_train)

accuracy = svm.score(term_docs_test, label_test)

print('The accuracy on testing set is: {0:.1f}%'.format(accuracy*100))
```

The accuracy on testing set is: 87.9%

다중 클래스 SVM in Python

- 클래스 별 정확도 평가

```
from sklearn.metrics import classification_report  
  
prediction = svm.predict(term_docs_test)  
  
report = classification_report(label_test, prediction)  
  
print(report)
```

	precision	recall	f1-score	support
0	0.79	0.77	0.78	319
1	0.91	0.94	0.92	389
2	0.98	0.95	0.97	399
3	0.91	0.92	0.92	394
4	0.73	0.74	0.73	251
avg / total	0.88	0.88	0.88	1752

Kernel in SVM

Kernel in SVM

- 앞서 지나갔던 SVM의 Kernel에 대해서 알아보자.

SVM 학습

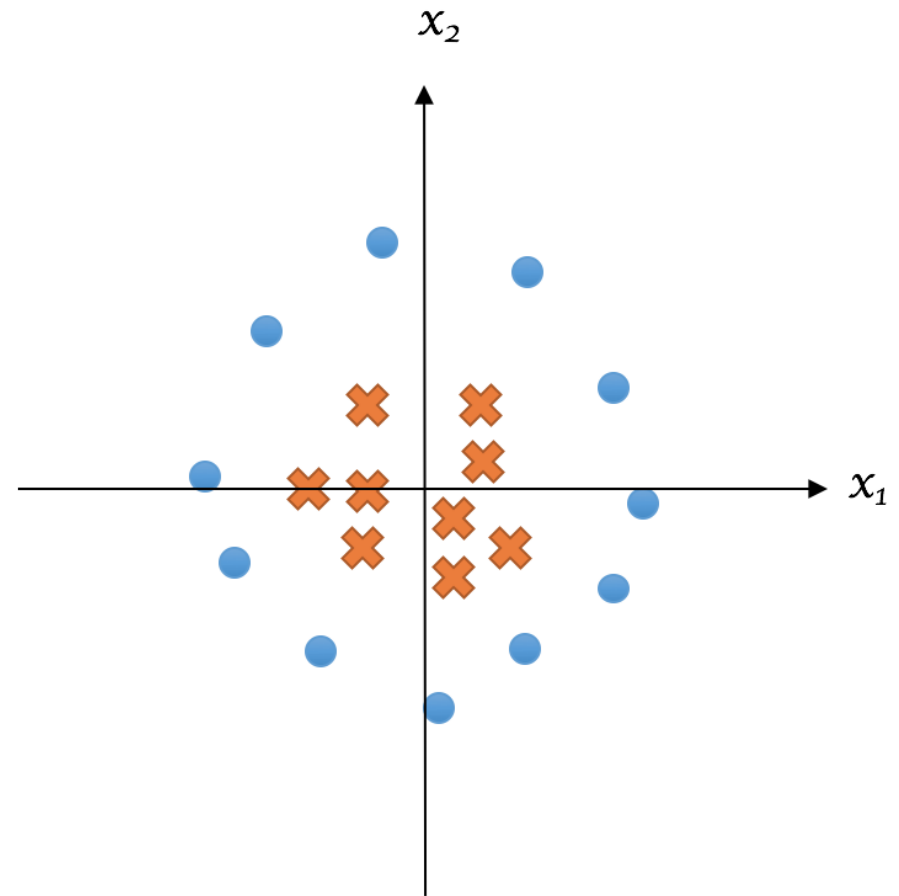
Kernel ?

```
from sklearn.svm import SVC  
svm = SVC(kernel='linear', C=1.0, random_state=42)  
svm.fit(term_docs_train, label_train)
```

```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,  
    decision_function_shape='ovr', degree=3, gamma='auto', kernel='linear',  
    max_iter=-1, probability=False, random_state=42, shrinking=True,  
    tol=0.001, verbose=False)
```

비선형 문제 Non-Linear Problem

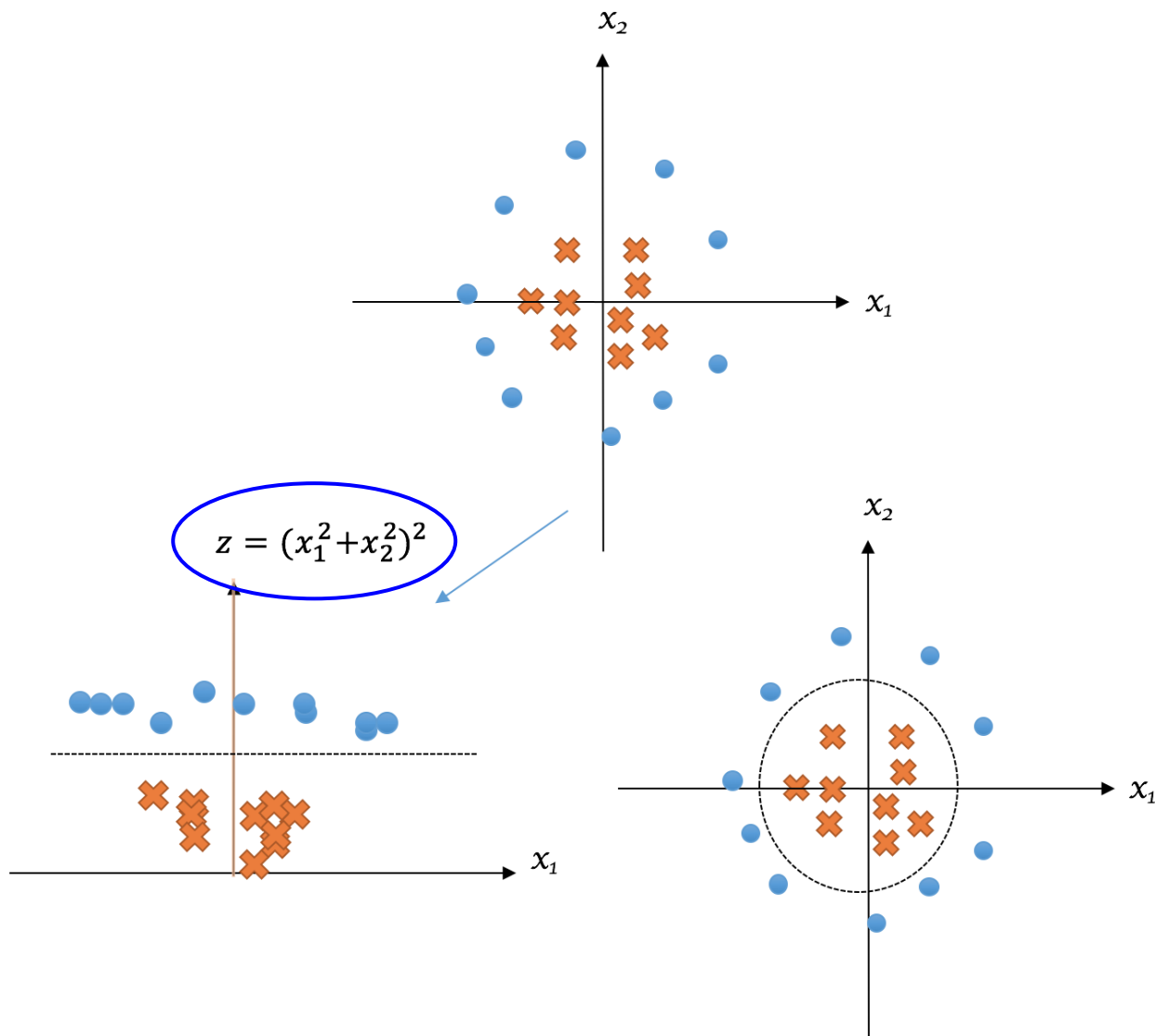
- 아래와 같이 어떤 데이터들은 **선형 식**으로 완벽하게
잘 분류해 낼 수 없다.
- 선형으로 모델링 할 수
없는 문제를
비선형 문제라고 한다.



SVM과 비선형 문제

- 기본적으로 SVM은 선형식으로 표현되는 하이퍼플레인에 기반하고 있어 비선형 문제에 적합하지 않다.
- 하지만 선형으로 양분할 수 있도록 피쳐 공간을 변환(Transform)한 뒤에 SVM을 적용하는 것이 가능하다.

고차원 피쳐 공간으로 변환



대표 커널 함수

- 다항식 커널

$$K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z} + 1)^p$$

- RBF 커널

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(\frac{-\|\mathbf{x} - \mathbf{z}\|}{2\sigma^2}\right)$$

- 하이퍼볼릭 탄젠트 커널

$$K(\mathbf{x}, \mathbf{z}) = \tanh(\alpha \mathbf{x} \cdot \mathbf{z} + \beta)$$

RBF Kernel Radial Basis Function

- 대표적인 SVM 커널이며, 가우시안 커널이라고도 불린다.
- RBF Kernel은 파라미터 γ 를 가지는데, 이 값에 따라서 변환 결과를 조절할 수 있다.
 - $\gamma = \frac{1}{2\sigma^2}$
 - γ 값이 크면 학습 데이터를 정확히 맞추는 변환 함수가 만들어져서 오버피팅 위험이 높아진다.
 - γ 값이 작으면 학습 데이터를 덜 정확히 맞추도록 변환 함수가 만들어져서 학습 데이터를 잘 대변하지 못할 수 있다.

Gamma for RBF Kernel

- 임의의 데이터를 만들어서 RBF Kernel의 Gamma 값 변화에 따른 결과 차이를 살펴보자.

```
import numpy as np
import matplotlib.pyplot as plt

# np.c_는 Array를 더 높은 차원 Array에 쌓아주는 함수

X = np.c_[# negative class
          (.3, -.8),
          (-1.5, -1),
          (-1.3, -.8),
          (-1.1, -1.3),
          (-1.2, -.3),
          (-1.3, -.5),
          (-.6, 1.1),
          (-1.4, 2.2),
          (1, 1),

          # positive class
          (1.3, .8),
          (1.2, .5),
          (.2, -2),
          (.5, -2.4),
          (.2, -2.3),
          (0, -2.7),
          (1.3, 2.1)].T

Y = [-1] * 8 + [1] * 8
gamma_option = [1, 2, 4]
```

Gamma for RBF Kernel

■ 그래프 출력하기

```
import matplotlib.pyplot as plt

plt.figure(1, figsize=(4*len(gamma_option), 4))

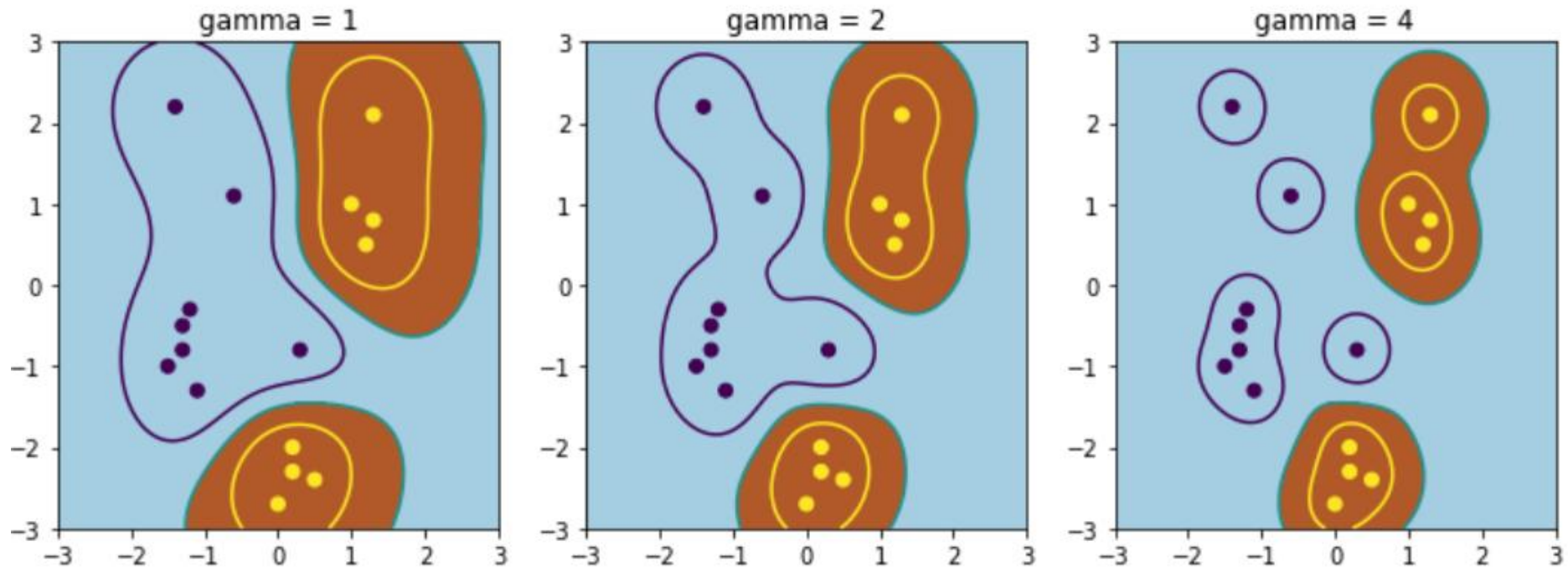
for i, gamma in enumerate(gamma_option, 1):
    svm = SVC(kernel='rbf', gamma=gamma)
    svm.fit(X, Y)
    plt.subplot(1, len(gamma_option), i)
    plt.scatter(X[:, 0], X[:, 1], c=Y, zorder=10)
    XX, YY = np.mgrid[-3:3:200j, -3:3:200j] #좌표 범위

    Z = svm.decision_function(np.c_[XX.ravel(), YY.ravel()])
    Z = Z.reshape(XX.shape)

    plt.pcolormesh(XX, YY, Z > 0, cmap=plt.cm.Paired) # 클래스 별 색칠
    plt.contour(XX, YY, Z, levels=[-.5, 0, .5]) # Levels은 경계선 범위 결정
    plt.title('gamma = %d' % gamma)
plt.show()
```

Gamma for RBF Kernel

- 그래프 출력하기



Linear Kernel과 RBF Kernel

- 기본적으로는 Linear Kernel의 사용을 추천하나, 아래와 같은 가이드라인을 참고할 수 있다.

Case	Linear	RBF
Expert prior knowledge	If linearly separable	If nonlinearly separable
Visualizable data of 1 to 3 dimension	If linearly separable	If nonlinearly separable
Both numbers of features and instances are large	First choice	
Features $\gg$ Instances	First choice	
Instances $\gg$ Features	First choice	
Others		First choice

SVM – Topic Classification

온라인 포스트 토픽 분류 문제

- 이제 Newsgroup20 데이터에서 20개 주제를 모두 고려하는 포스트 토픽 분류기를 만들어보자.
 - 앞서 만든 것은 5개 주제 클래스까지 했었다.
- 더불어 다양한 하이퍼 파라미터(예: C)들을 어떻게 최적화 할 수 있을지 알아 보겠다.

온라인 포스트 토픽 분류 문제

(1) 데이터 불러오기 및 전처리

```
data_train = fetch_20newsgroups(subset='train',  
                                categories=None, random_state=42)  
  
data_test = fetch_20newsgroups(subset='test',  
                               categories=None, random_state=42)  
  
cleaned_train = clean_text(data_train.data)  
label_train = data_train.target  
  
cleaned_test = clean_text(data_test.data)  
label_test = data_test.target  
  
term_docs_train = tfidf_vectorizer.fit_transform(cleaned_train)  
term_docs_test = tfidf_vectorizer.transform(cleaned_test)
```

온라인 포스트 토픽 분류 문제

(2) SVM 모델 객체 생성

```
svc_libsvm = SVC(kernel='linear')
```

(3) 하이퍼 파라미터 찾기

- 다양한 하이퍼 파라미터 값 (예: $C = 0.1, 1, 10, 100$) 각각의 경우에 대해서 교차검증을 수행해 가장 높은 정확도를 만드는 값을 탐색
- For 문을 이용해 직접 코드를 작성하는 것도 가능하지만 사이킷런에서 제공하는 `GridSearchCV`를 사용하면 간결하게 수행 가능

GridSearchCV 사용하기

■ GridSearchCV 객체 생성

```
from sklearn.model_selection import GridSearchCV  
  
parameters = {'C': (0.1, 1, 10, 100)}  
  
grid_search = GridSearchCV(svc_libsvm, parameters, n_jobs=-1, cv=3)
```

- ✓ svc_libsvm : 검증에 사용할 모델 객체
- ✓ parameters : 테스트하고자 하는 하이퍼 파라미터
- ✓ n_jobs : 병렬 수행에 사용할 코어 수 (-1은 가능한 것 모두 사용)
- ✓ cv : 교차검증 k 값

GridSearchCV 사용하기

■ GridSearchCV 수행

- C값으로 0.1, 1, 10, 100을 사용하는 경우에 대해서 각각 3-folds cross validation을 수행

```
import timeit
start_time = timeit.default_timer()

grid_search.fit(term_docs_train, label_train)

print("--- %0.3fs seconds ---" % (timeit.default_timer() - start_time))
```

```
--- 157.210s seconds ---
```

```
grid_search.best_params_
```

```
{'C': 10}
```

```
grid_search.best_score_
```

```
0.8577868127983029
```

뉴스 포스트 토픽 분류 문제

(4) 테스트 데이터에 대한 최종 성능 평가

- $C = 10$ 을 사용해서 학습한 모델 사용

```
svc_libsvm_best = grid_search.best_estimator_  
accuracy = svc_libsvm_best.score(term_docs_test, label_test)  
print('The accuracy on testing set is: {0:.1f}%'.format(accuracy*100))
```

The accuracy on testing set is: 75.4%

더 다양한 파이프라인 고려하기

- 선형 SVM의 학습 파라미터 C만 고려했지만
이밖에도 학습에 관련된 다양한 세팅이 존재한다.
- 예) 피처 추출 : TFIDF 벡터화 할 때 사용되는 파라미터
 - TfidfVectorizer의 파라미터들
 - max_df: 문서에서 일반적으로 발견되는 용어를 배제하기 위한 문서 빈도 최대 값
(전체 문서 수 중 등장 문서 수 비율)
 - max_features: 가장 중요하게 고려할 피처의 개수 (지금까지는 8000개 사용)
 - sublinear_tf: 로그 함수 또는 다른 함수 등을 이용해 용어의 출현 빈도를 변환
 - ...

더 다양한 파이프라인 고려하기

- 이처럼 여러 파라미터가 존재할 때 parameter_pipeline을 만들어 GridSearchCV를 수행하면 편리

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(stop_words='english')),
    ('svc', SVC(kernel='linear'))])
```

```
parameters_pipeline = {
    'tfidf__max_df': (0.25, 0.5),
    'tfidf__max_features': (40000, 50000),
    'tfidf__sublinear_tf': (True, False),
    'svc__C': (0.1, 1, 10, 100),
}
```


더 다양한 파이프라인 고려하기

```
grid_search = GridSearchCV(pipeline, parameters_pipeline, n_jobs=-1, cv=3)

start_time = timeit.default_timer()
grid_search.fit(cleaned_train, label_train)

print("--- %0.3fs seconds ---" % (timeit.default_timer() - start_time))

--- 464.497s seconds ---
```

```
grid_search.best_params_
```

```
{'svc__C': 10,
 'tfidf__max_df': 0.25,
 'tfidf__max_features': 40000,
 'tfidf__sublinear_tf': True}
```

```
grid_search.best_score_
```

```
0.8831536149902776
```

더 다양한 파이프라인 고려하기

- 최적의 파라미터로 테스트 데이터에 대한 성능 평가

```
pipeline_best = grid_search.best_estimator_  
  
accuracy = pipeline_best.score(cleaned_test, label_test)  
print('The accuracy on testing set is: {0:.1f}%'.format(accuracy*100))
```

The accuracy on testing set is: 78.9%

앞서 파라미터 C만 GridSearch 한 것보다

성능이 조금 더 향상된 것을 볼 수 있다.

SVM – Fetal State Classification

심전도 데이터로 태아 상태 분류

- RBF 커널을 사용하는 SVM을 실습해 보자.

< 태아 상태 분류 문제 >

산부인과 의사가 심전도(Cardiotocograms) 데이터를 세 가지 태아 상태 (정상, 의심, 아픔) 중 하나로 분류하는 데 도움이 되도록 분류기를 만들고자 한다.

데이터

- UCI 머신러닝 데이터 저장소에서 다운로드
 - <https://archive.ics.uci.edu/ml/machine-learning-databases/00193/CTG.xls>
- 데이터는 태아 심장 박동에 관한 측정값들, 자궁 수축 정도에 관한 측정값들을 포함하고 있다.

	FileName	Date	SegFile	b	e	LBE	LB	AC	FM	UC	...	C	D	E	AD	DE	LD	FS	SUSP	CLASS	NSP
0	NaN	NaT	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	Variab10.txt	1996-12-01	CTG0001.txt	240.0	357.0	120.0	120.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	9.0	2.0
2	Fmcs_1.txt	1996-05-03	CTG0002.txt	5.0	632.0	132.0	132.0	4.0	0.0	4.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
3	Fmcs_1.txt	1996-05-03	CTG0003.txt	177.0	779.0	133.0	133.0	2.0	0.0	5.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
4	Fmcs_1.txt	1996-05-03	CTG0004.txt	411.0	1192.0	134.0	134.0	2.0	0.0	6.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0

- 처음 4번째 컬럼 부터 마지막에서 세번째 컬럼까지가 피쳐
- 마지막 컬럼은 클래스 레이블

데이터 읽기 & 학습/테스트 데이터 나누기

■ 데이터 불러오기

- 피쳐들을 X에 클래스 레이블을 Y에 저장
- Counter 함수를 통해 클래스 별 샘플 수를 확인 가능

```
X = df.iloc[1:2126, 3:-2].values  
Y = df.iloc[1:2126, -1].values
```

```
from collections import Counter  
Counter(Y)
```

```
Counter({2.0: 295, 1.0: 1654, 3.0: 176})
```

■ 학습 / 테스트 데이터 나누기

```
from sklearn.model_selection import train_test_split  
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,  
                                                    test_size=0.2, random_state=42)
```

모델 학습 및 파라미터 탐색

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
import timeit

svc = SVC(kernel='rbf')
parameters = {'C': (100, 1e3, 1e4, 1e5),
              'gamma': (1e-08, 1e-7, 1e-6, 1e-5)}

grid_search = GridSearchCV(svc, parameters, n_jobs=-1, cv=3)

start_time = timeit.default_timer()
grid_search.fit(X_train, Y_train)

print("--- %0.3fs seconds ---" % (timeit.default_timer() - start_time))

--- 21.606s seconds ---
```

최종 파라미터 확인 & 최종 정확도 확인

```
grid_search.best_params_
```

```
{'C': 100000.0, 'gamma': 1e-07}
```

```
grid_search.best_score_
```

```
0.9447058823529412
```

```
svc_best = grid_search.best_estimator_  
accuracy = svc_best.score(X_test, Y_test)  
print('The accuracy on testing set is: {0:.1f}%'.format(accuracy*100))
```

```
The accuracy on testing set is: 95.5%
```


클래스 별 성능 확인

```
from sklearn.metrics import classification_report  
  
prediction = svc_best.predict(X_test)  
report = classification_report(Y_test, prediction)  
print(report)
```

	precision	recall	f1-score	support
1.0	0.96	0.98	0.97	324
2.0	0.89	0.91	0.90	65
3.0	1.00	0.78	0.88	36
avg / total	0.96	0.96	0.95	425

Lab 3-1: SVM

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.

Decision Tree

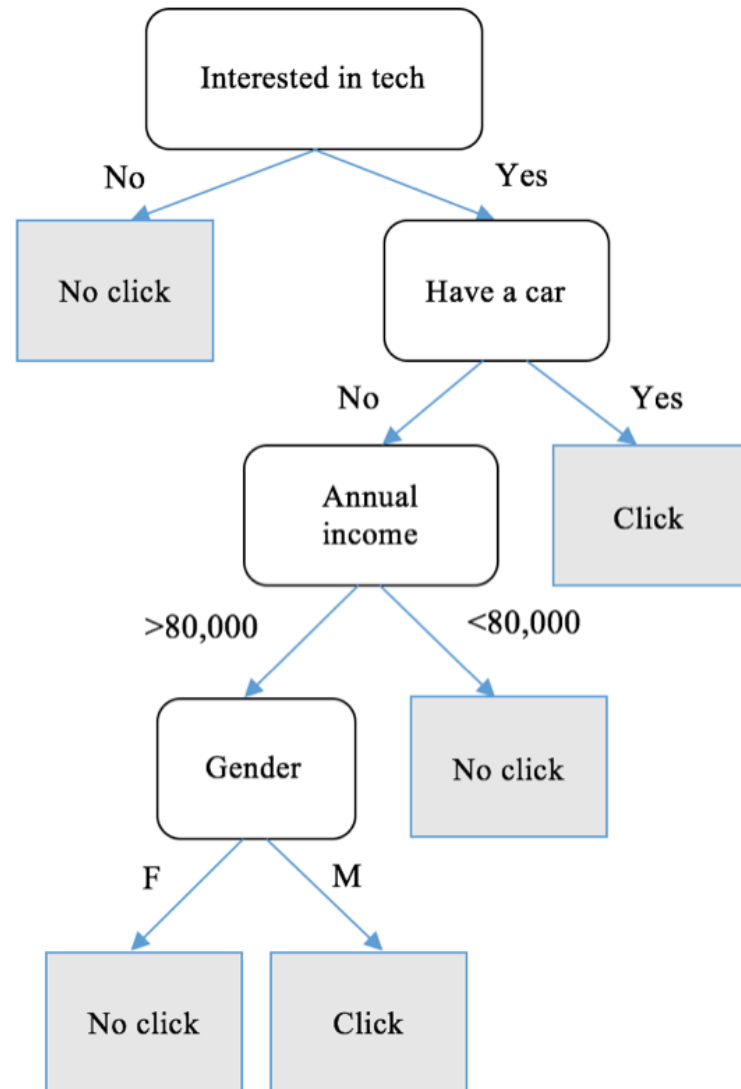
의사결정 트리 분류기 Decision Tree Classifier

- 의사결정 트리 분류기는 **트리 모양의 순차형 다이어그램**을 통해 주어진 데이터를 분류
 - 트리의 루트부터 시작해서 모든 중간 노드들은 의사 결정 사항(조건)
 - 트리의 맨 끝에 있는 리프 노드는 의사 결정 사항에 따른 최종 결과
- 의사결정 트리 분류기의 동작은 **의사결정 트리를 생성하는 학습 단계**와 **생성된 의사결정 트리에 따라 주어진 데이터를 분류**하는 분류 단계로 나뉘 볼 수 있다.
 - 대표적인 트리 생성 알고리즘 : ID3, C4.5, **CART**, CHAID

의사결정 트리 예

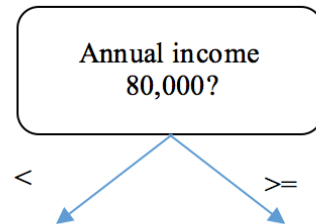
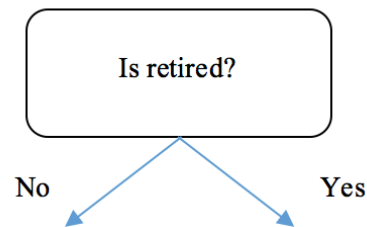
User gender	Annual income	Have a car	Interested in tech	Click
M	200,000	True	True	1
F	5,000	False	False	0
F	100,000	True	True	1
M	10,000	True	False	0
M	80,000	False	False	0
...	...	...	...	...
...	...	...	...	...

M	120,000	True	True	?
F	70,000	False	True	?

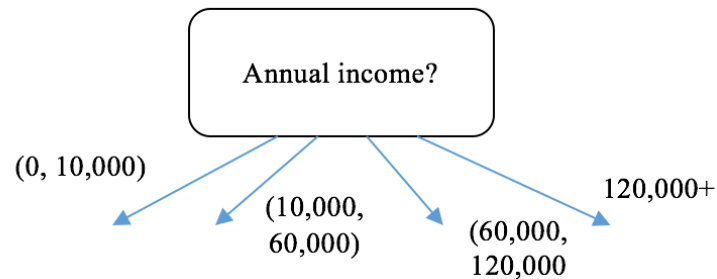
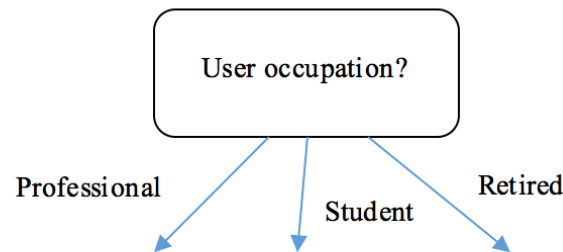


의사결정 트리 생성: 파티셔닝

- 의사결정 트리는 데이터를 재귀적으로 파티셔닝 하여 생성



Binary splitting

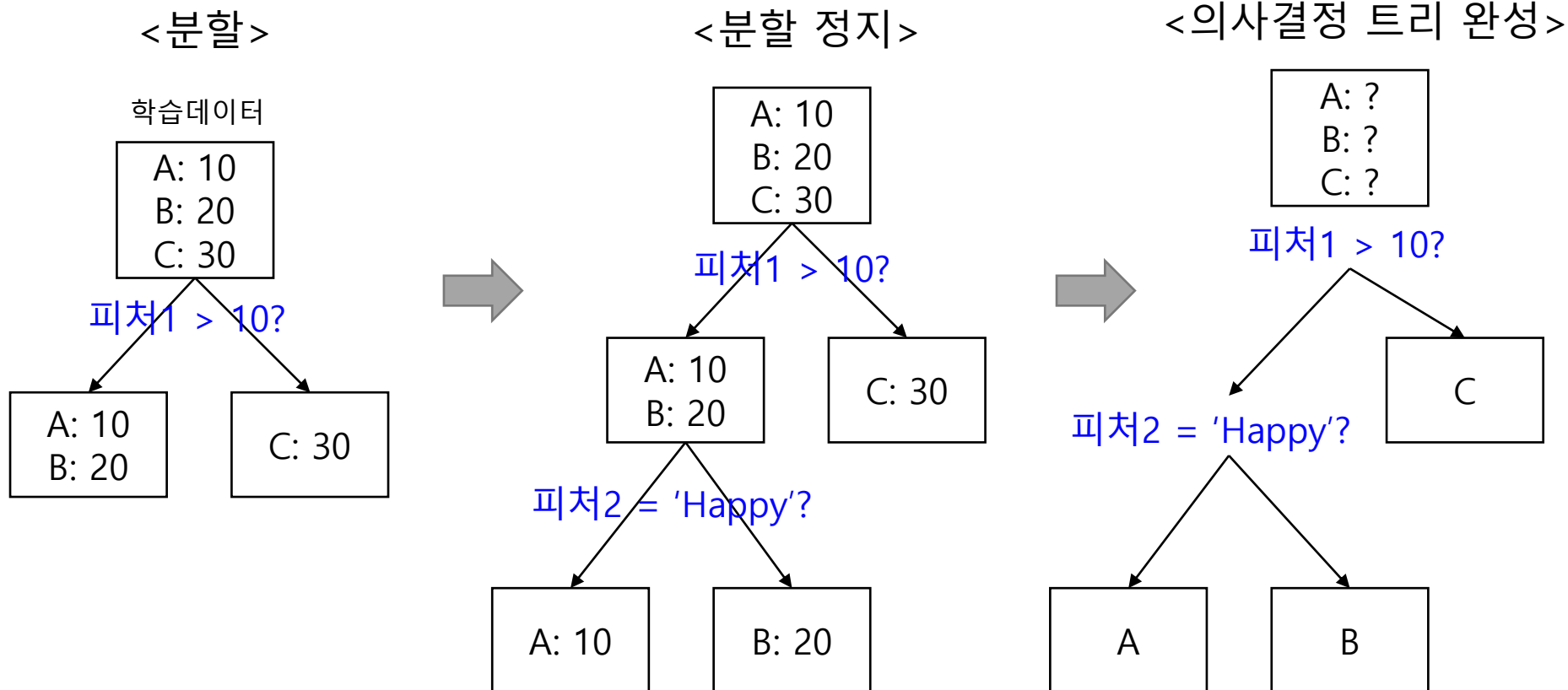


Multiway splitting

< 파티셔닝 예 >

CART Classification and Regression Tree

- 학습데이터를 사용하여 각 노드를 **왼쪽 자식** 노드와 **오른쪽 자식** 노드로 분할해 가며 트리를 생성



CART : 분할

- 기본적으로는 클래스 별로 데이터를 잘 가를 수 있으면 좋은 분할
- 하지만 하나의 피처와 해당 값으로 전체 데이터를 클래스 별로 완벽하게 가를 수는 없기 때문에, 단계적으로 분할 조건들을 탐색해 나가야 한다.
- CART는 각 노드에서 분할을 위한 **피처**와 해당 **피처의 값**의 조합을 탐욕적(Greedy) 방식으로 탐색
 - ➔ Global Optima가 아니라 Local Optima를 우선 고려

이를 위해서는 어떤 피처와 해당 값을 선택했을 때

데이터를 클래스 별로 얼마나 잘 분할해 내는지 평가할 수 있는 값이 필요하다.

트리 분할 측정 척도

- 각 피처와 해당 값들이 가장 잘 조합된 결과를 선택하기 위해 분할이 얼마나 잘 이루어졌는지를 측정하는 척도
- 대표적인 트리 분할 측정 척도
 - 지니 계수 Gini Impurity
 - 정보 이득 Information Gain

DT - Gini

트리 분할 측정 척도 : 지니 계수

- 데이터가 클래스 별로 고르게 분포되어 있는지, 아니면 특정 클래스에 편중되어 분포되어 있는지 나타냄
 - 편중될 수록 값이 작고, 고를 수록 값이 크다 (Impurity)

$$Gini = 1 - \sum_{K=1}^K f_k^2 \quad (f_k \text{는 } k\text{번째 클래스에 해당하는 데이터의 비율})$$

두 개 클래스가 존재할 때,

- 모든 데이터가 한 클래스에만 속한다면 지니계수는 $1 - (1^2 + 0^2) = 0$
- 두 클래스에 반반씩 속한다면 지니계수는 $1 - (0.5^2 + 0.5^2) = 0.5$

트리 분할 측정 척도 : 지니 계수

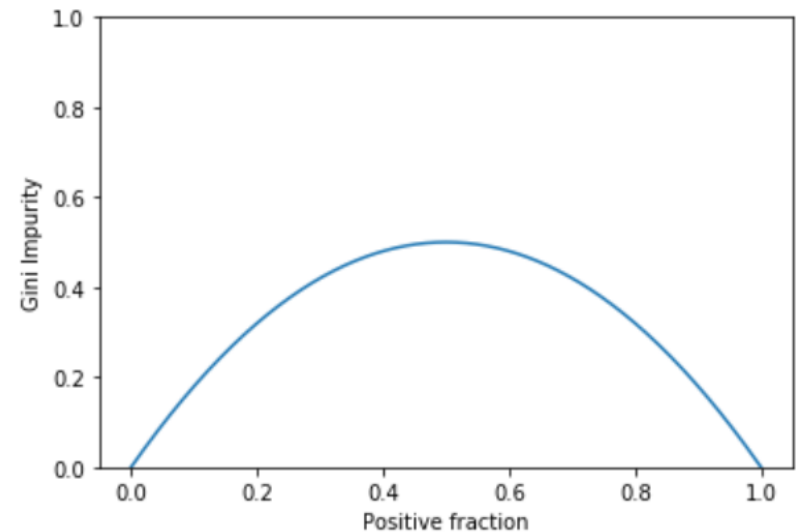
■ 지니 계수 예시

- 음성 vs. 양성 클래스
- 양성 클래스 비율에 따라서 지니 계수가 어떻게 달라지는지 확인

```
import matplotlib.pyplot as plt
import numpy as np

pos_fraction = np.linspace(0.00, 1.00, 1000)

gini = 1 - pos_fraction**2 - (1-pos_fraction)**2
plt.plot(pos_fraction, gini)
plt.ylim(0, 1)
plt.xlabel('Positive fraction')
plt.ylabel('Gini Impurity')
plt.show()
```



트리 분할 측정 척도 : 지니 계수

■ 지니 계수 함수 구현

```
def gini_impurity(labels):  
    # When the set is empty, it is also pure  
    if not labels:  
        return 0  
    # Count the occurrences of each label  
    counts = np.unique(labels, return_counts=True)[1]  
  
    fractions = counts / float(len(labels))  
  
    return 1 - np.sum(fractions ** 2)
```

```
print('{0:.4f}'.format(gini_impurity([1, 1, 0, 1, 0])))
```

0.4800

```
print('{0:.4f}'.format(gini_impurity([1, 1, 0, 1, 0, 0])))
```

0.5000

```
print('{0:.4f}'.format(gini_impurity([1, 1, 1, 1])))
```

0.0000

 np.unique(array)는 기본적으로 array의 고유한 성분값들을 반환

- 예: [1, 1, 2] → [1, 2]

만약 return_counts=True 설정하면 고유한 성분값들 Array와 각 성분값에 해당하는 성분 수 Array가 Tuple로 반환

- 예: [1, 1, 2] → ([1, 2], [2, 1])

트리 분할 측정 척도 : 지니 계수

- 분할한 결과로 만들어진 자식 노드들 각각에 대해서 지니 계수를 구하고 가중 평균 하여 분할 결과를 측정
 - 이 값이 작으면 작을 수록 클래스를 편중해서 잘 갈라냈음을 의미

$$\sum_{c=1}^C \frac{n_c}{n_p} Gini_c$$

C : 자식 노드 집합

$Gini_c$: 자식 노드 c 에 있는 데이터의 지니계수

n_p : 부모 노드 p 에 속하는 데이터 수

n_c : 자식 노드 c 에 속하는 데이터 수

트리 분할 측정 척도 : 지니 계수

User gender	Interested in tech	Click	Group by gender
M	True	1	Group 1
F	False	0	Group 2
F	True	1	Group 2
M	False	0	Group 1
M	False	1	Group 1

#1 split based on gender

User gender	Interested in tech	Click	Group by interest
M	True	1	Group 1
F	False	0	Group 2
F	True	1	Group 1
M	False	0	Group 2
M	False	1	Group 2

#2 split based on interest in tech

$$\text{\#1의 지니계수} = \frac{3}{5} \left[1 - \left(\left(\frac{2}{3} \right)^2 + \left(\frac{1}{3} \right)^2 \right) \right] + \frac{2}{5} \left[1 - \left(\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right) \right] = 0.467$$

$$\text{\#2의 지니계수} = \frac{2}{5} \left[1 - ((1)^2 + (0)^2) \right] + \frac{3}{5} \left[1 - \left(\left(\frac{1}{3} \right)^2 + \left(\frac{2}{3} \right)^2 \right) \right] = 0.267$$

성별보다 기술 관심도를 기준으로 분할 하는 것이 더 좋다.

DT - Entropy

트리 분할 측정 척도 : 정보 이득

- 트리 분할 후 얼마나 순도가 더 좋아졌는지 평가
= 트리 분할을 통해 불확실성이 얼마나 줄어드는지 측정
(전 vs. 후를 비교하는 것이 특징)
 - 불확실성이 높다는 것은 클래스 별로 고르게 분포되어 있다는 것
 - 모든 클래스에 똑같은 비율로 데이터가 존재할 때, 여기서 어떤 데이터를 임의로 뽑는다면 이 데이터가 어떤 클래스에 속할지 불확실하다.
 - 반대로 특정 클래스에 편중되어 있어 데이터의 클래스에 대한 불확실성이 낮다.
- 정보 이득은 트리 분할 전 후의 엔트로피 값 차이로 계산된다.

트리 분할 측정 척도 : 정보 이득

■ 엔트로피 (Entropy)

$$Entropy = - \sum_{K=1}^K f_k * \log_2 f_k \quad (f_k \text{는 } k\text{번째 클래스에 해당하는 데이터의 비율})$$

두 개 클래스가 존재할 때,

- 모든 데이터 한 클래스에만 속한다면 엔트로피는 $-(1 * \log_2 1 + 0) = 0$
- 두 클래스에 반반씩 속한다면 엔트로피는 $-(0.5 * \log_2 0.5 + 0.5 * \log_2 0.5) = 1$

트리 분할 측정 척도 : 정보 이득

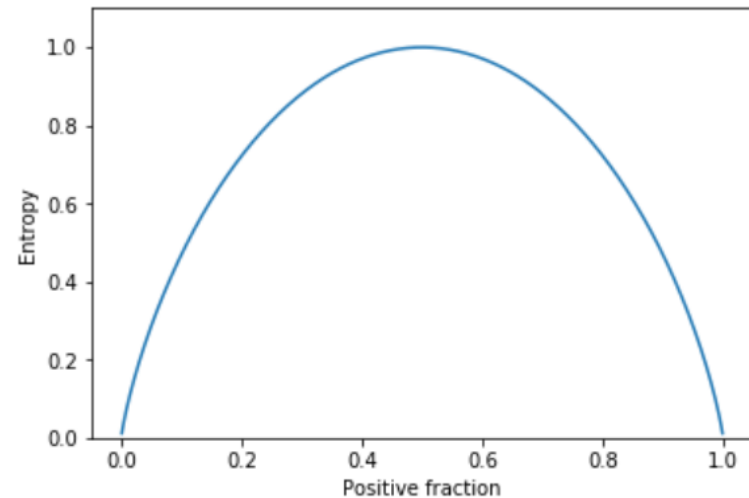
■ 엔트로피 예시

- 음성 vs. 양성 클래스
- 양성 클래스 비율에 따라서 엔트로피가 어떻게 달라지는지 확인

```
pos_fraction = np.linspace(0.00, 1.00, 1000)

ent = - (pos_fraction * np.log2(pos_fraction) +
        (1 - pos_fraction) * np.log2(1 - pos_fraction))

plt.plot(pos_fraction, ent)
plt.xlabel('Positive fraction')
plt.ylabel('Entropy')
plt.ylim(0, 1.1)
plt.show()
```



트리 분할 측정 척도 : 정보 이득

- 엔트로피 함수 구현

```
def entropy(labels):  
    if not labels:  
        return 0  
  
    counts = np.unique(labels, return_counts=True)[1]  
  
    fractions = counts / float(len(labels))  
  
    return - np.sum(fractions * np.log2(fractions))
```

```
print('{0:.4f}'.format(entropy([1, 1, 0, 1, 0])))
```

0.9710

```
print('{0:.4f}'.format(entropy([1, 1, 0, 1, 0, 0])))
```

1.0000

```
print('{0:.4f}'.format(entropy([1, 1, 1, 1])))
```

-0.0000

트리 분할 측정 척도 : 정보 이득

- 정보이득은 트리 분할 전(부모 노드) 엔트로피와 트리 분할 후(자식 노드들) 엔트로피 값의 차이로 정의
 - 자식 노드들의 엔트로피 값은 지니 계수와 같은 방식으로 가중 평균하여 계산

$$Information\ Gain = Entropy(p) - \sum_c \frac{n_c}{n_p} Entropy(c)$$

트리 분할 측정 척도 : 정보 이득

User gender	Interested in tech	Click	Group by gender
M	True	1	Group 1
F	False	0	Group 2
F	True	1	Group 2
M	False	0	Group 1
M	False	1	Group 1

#1 split based on gender

User gender	Interested in tech	Click	Group by interest
M	True	1	Group 1
F	False	0	Group 2
F	True	1	Group 1
M	False	0	Group 2
M	False	1	Group 2

#2 split based on interest in tech

< 자식 엔트로피 >

$$\text{\#1의 엔트로피} = \frac{3}{5} \left[- \left(\frac{2}{3} * \log_2 \frac{2}{3} + \frac{1}{3} * \log_2 \frac{1}{3} \right) \right] + \frac{2}{5} \left[- \left(\frac{1}{2} * \log_2 \frac{1}{2} + \frac{1}{2} * \log_2 \frac{1}{2} \right) \right] = 0.951$$

$$\text{\#2의 엔트로피} = \frac{2}{5} \left[- (1 * \log_2 1 + 0) \right] + \frac{3}{5} \left[- \left(\frac{1}{3} * \log_2 \frac{1}{3} + \frac{2}{3} * \log_2 \frac{2}{3} \right) \right] = 0.551$$

트리 분할 측정 척도 : 정보 이득

User gender	Interested in tech	Click	Group by gender
M	True	1	Group 1
F	False	0	Group 2
F	True	1	Group 2
M	False	0	Group 1
M	False	1	Group 1

#1 split based on gender

User gender	Interested in tech	Click	Group by interest
M	True	1	Group 1
F	False	0	Group 2
F	True	1	Group 1
M	False	0	Group 2
M	False	1	Group 2

#2 split based on interest in tech

$$\text{부모 엔트로피} = -\left(\frac{3}{5} * \log_2 \frac{3}{5} + \frac{2}{5} * \log_2 \frac{2}{5}\right) = 0.971$$

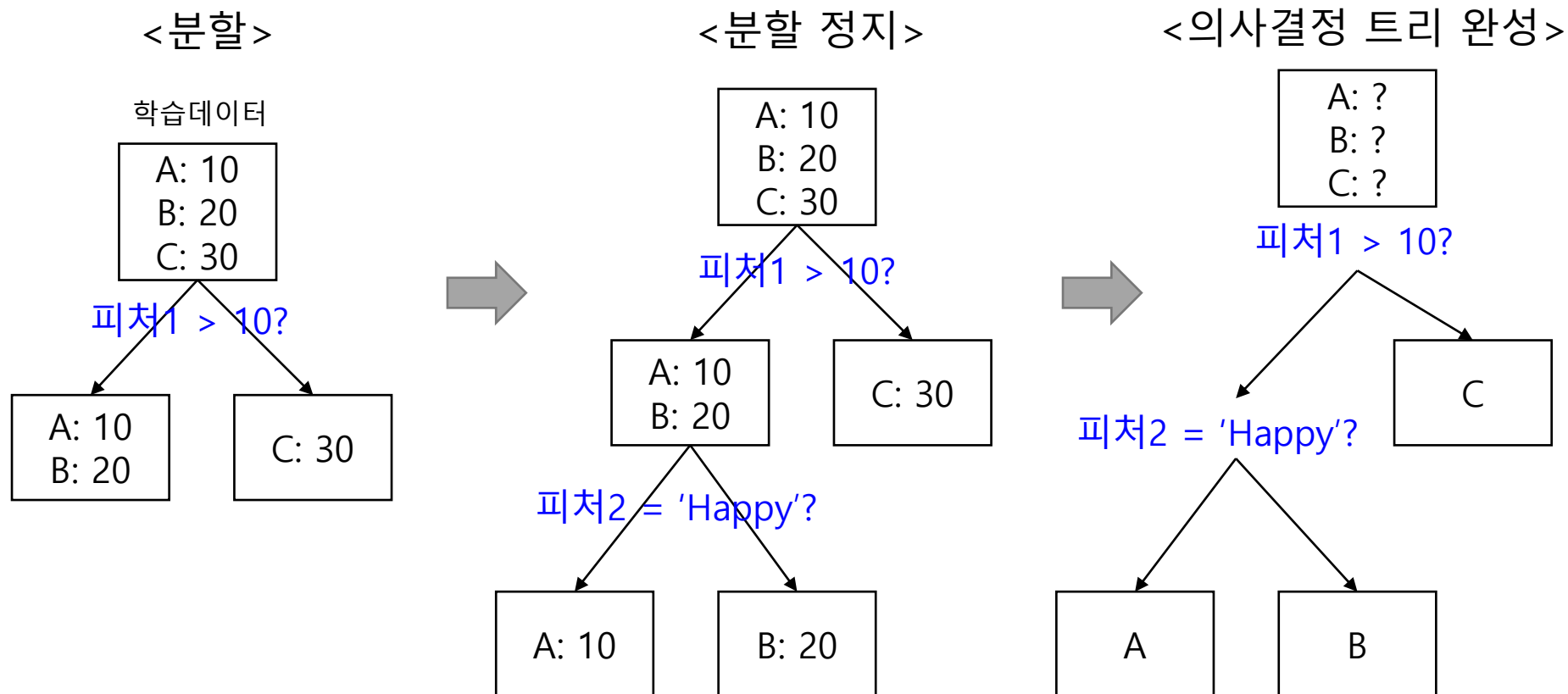
#1의 정보이득 = $0.971 - 0.951 = 0.020$ #2의 정보이득 = $0.971 - 0.551 = 0.420$

성별보다 기술 관심도를 기준으로 분할 하는 것이 더 좋다.

CART

CART Classification and Regression Tree

- 학습데이터를 사용하여 각 노드를 **왼쪽 자식** 노드와 **오른쪽 자식** 노드로 분할해 가며 트리를 생성



CART Classification and Regression Tree

- 분할 : 각 파티션 단계에서 가장 중요한 피처와 해당 피처의 값의 조합을 탐욕적(Greedy) 방식으로 탐색
 - ➔ Global Optima가 아니라 Local Optima를 우선 고려
- 정지 : 분할 과정은 다음과 같은 조건이 만족되면 중지
 - 분할 후 만들어질 노드의 샘플 수가 일정 수준 이하일 때
 - 트리의 깊이가 일정 수준 이상일 때
 - ➔ 오버피팅을 피하기 위한 조치
- 분할 되지 않은 마지막 노드들에 클래스 레이블을 할당하는데, 각 노드에 있는 데이터의 클래스 중 가장 다수 클래스를 할당

CART 동작 예

- 다음 데이터에 대해서 CART 알고리즘을 적용해 보자.
 - 데이터는 Interest (사용자 관심사), Occupation (직업), Click (광고를 클릭 했는지 여부)로 구성되어 있으며, Click은 클래스 변수다.
 - 분할 측정 기준으로는 지니 계수를 사용해 보자.
 - 노드 당 샘플 수가 최소 2 이상이 되도록 하자.

Interest	Occupation	Click
Tech	Professional	1
Fashion	Student	0
Fashion	Professional	0
Sports	Student	0
Tech	Student	1
Tech	Retired	0
Sports	Professional	1

CART 동작 예 : 분할 #1

가능한 모든 조합에 대해서 지니 계수를 계산한다.

- $Gini(Interest, Tech)$
- $Gini(Interest, Fashion)$
- $Gini(Interest, Sports)$
- $Gini(Occupation, Professional)$
- $Gini(Occupation, Student)$
- $Gini(Occupation, Retired)$

Interest	Occupation	Click
Tech	Professional	1
Fashion	Student	0
Fashion	Professional	0
Sports	Student	0
Tech	Student	1
Tech	Retired	0
Sports	Professional	1

CART 동작 예 : 분할 #1

$$\text{Gini}(\text{Interest}, \text{Tech}) = \frac{3}{7} * \left(1 - \left(\left(\frac{2}{3} \right)^2 + \left(\frac{1}{3} \right)^2 \right) \right) + \frac{4}{7} * \left(1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) \right)$$

= 0.405

Interest	Occupation	Click
Tech	Professional	1
Fashion	Student	0
Fashion	Professional	0
Sports	Student	0
Tech	Student	1
Tech	Retired	0
Sports	Professional	1

Interest = Tech?

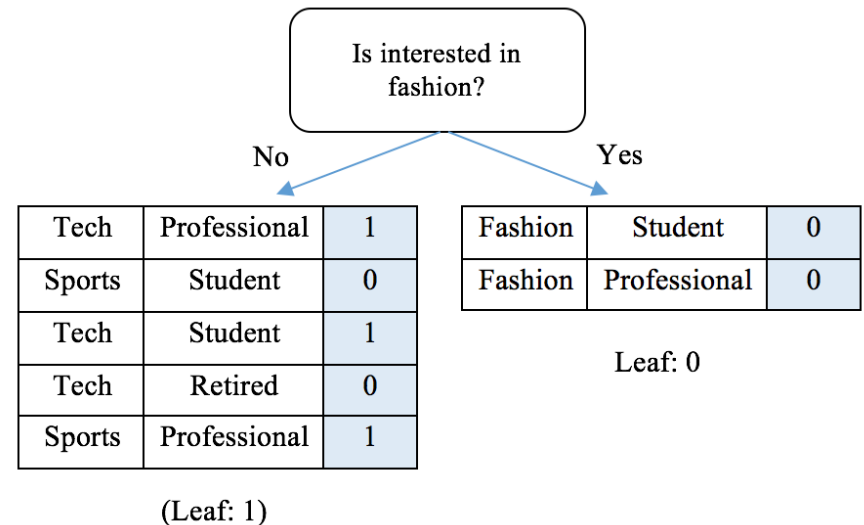


Interest	Occupation	Click
Tech	Professional	1
Tech	Student	1
Tech	Retired	0

Interest	Occupation	Click
Fashion	Student	0
Fashion	Professional	0
Sports	Student	0
Sports	Professional	1

CART 동작 예 : 분할 #1

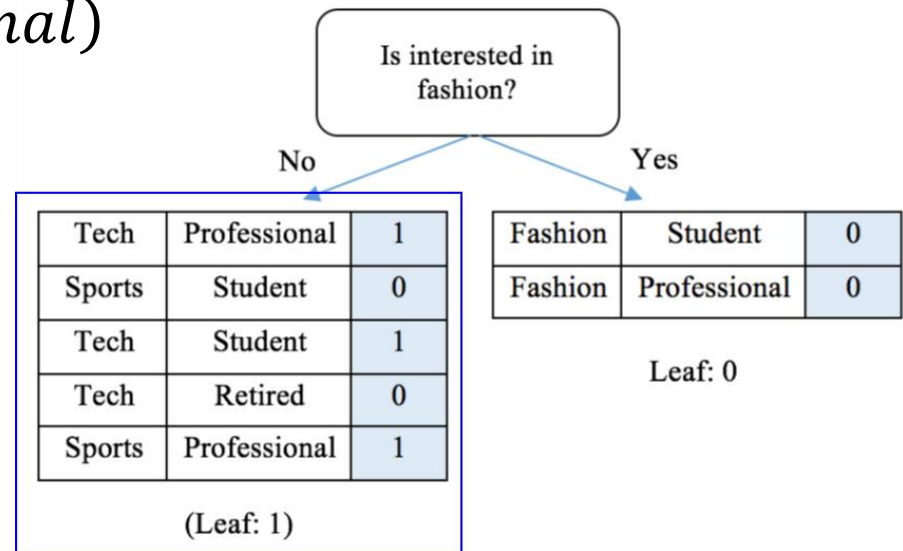
- $Gini(Interest, Tech) = 0.405$
- $Gini(Interest, Fashion) = 0.343$
- $Gini(Interest, Sports) = 0.486$
- $Gini(Occupation, Professional) = 0.405$
- $Gini(Occupation, Student) = 0.405$
- $Gini(Occupation, Retired) = 0.429$



CART 동작 예 : 분할 #2

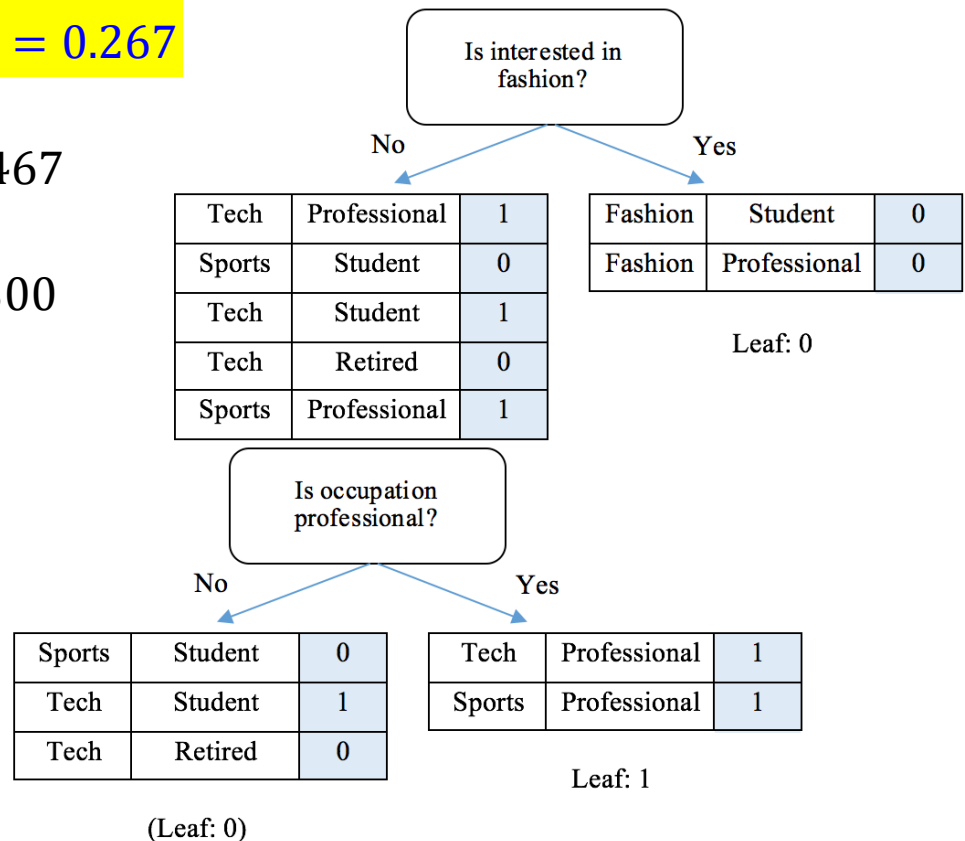
왼쪽 자식 노드에서 분할을 수행

- $Gini(Interest, Tech)$
- $Gini(Interest, Sports)$
- $Gini(Occupation, Professional)$
- $Gini(Occupation, Student)$
- $Gini(Occupation, Retired)$



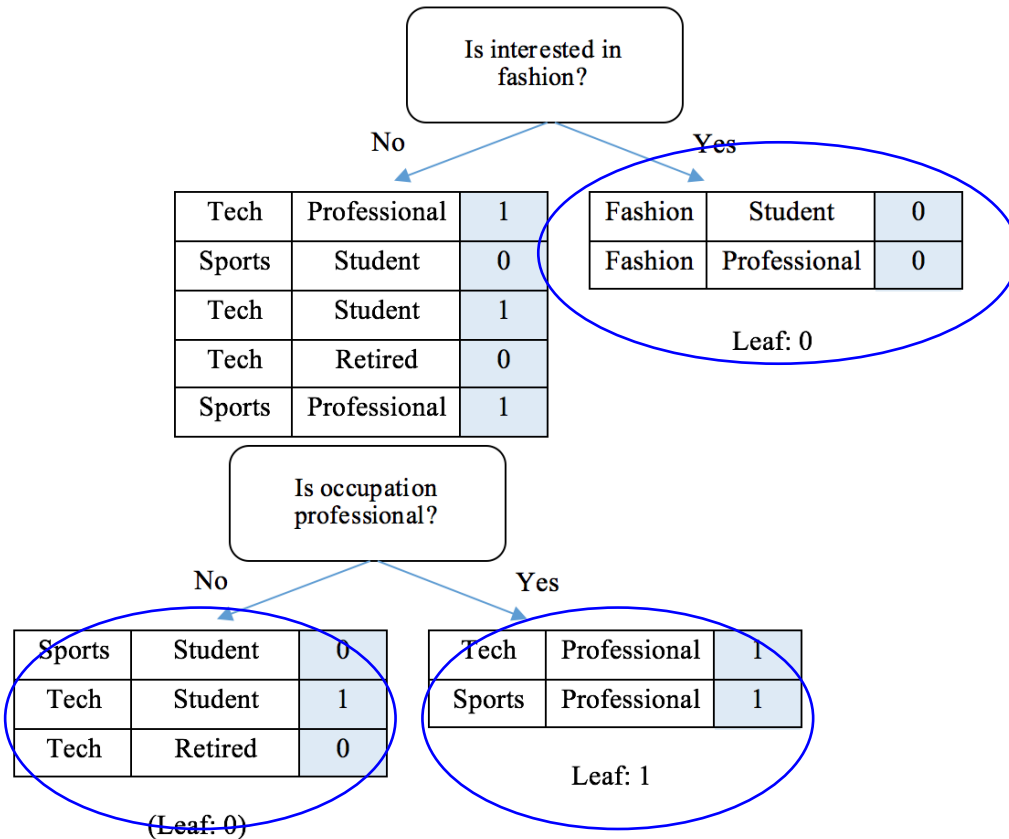
CART 동작 예 : 분할 #2

- $Gini(Interest, Tech) = 0.467$
- $Gini(Interest, Sports) = 0.467$
- $Gini(Occupation, Professional) = 0.267$
- $Gini(Occupation, Student) = 0.467$
- $Gini(Occupation, Retired) = 0.300$



CART 동작 예 : 분할 정지 및 완성

노드당 샘플 수 최소값으로 2를 사용하기 때문에, 더 이상 분할하지 않는다.



DT - Ensemble Method

모델의 조합 Ensemble Method

- 여러 모델을 독립적으로 학습 시킨 다음에
각 모델이 만들어 낸 결과를 조합하여 최종 결과를 생성
 - 배깅 (Bagging)
 - 부스팅 (Boosting)
 - 스택킹 (Stacking)
 - 블렌딩 (Blending)
 - 보팅 (Voting) 및 평균화 (Averaging)

배깅 Bagging

- 부트스트랩 집계(Bootstrap Aggregating)이라고도 함
- 하나의 학습 알고리즘이지만 여러 다른 데이터로 학습된 결과로 여러 모델을 만들고 이들의 출력 결과를 조합
- 단계
 - Replace 가능한 샘플링 방법으로 원본 학습 데이터에서 새로운 학습 데이터를 생성
 - 만들어진 학습 데이터 각각에 모델을 학습
 - 테스트 데이터에 대해서 학습된 모델들이 만들어낸 결과를 평균화 또는 최다 득표 보팅을 통해 최종 결과 생성

랜덤 포레스트 Random Forest

- 여러 개 의사 결정 트리 분류기 모델을 조합해서 사용하는 배깅 방법
 - 원본 데이터에서 무작위로 데이터를 추출하여 여러 개 학습 데이터 세트를 구축
 - 각 데이터로 모델을 학습 시켜, 여러 개 학습 모델을 생성
 - 새로운 데이터가 들어오면 개별 학습 모델이 판단한 결과들을 통합하여 다수 모델이 선택한 결과를 최종 결과로 선택
- 모든 피쳐들을 똑같이 다 사용하게 되면, 랜덤 포레스트에서 사용하는 개별 모델이 비슷해 지기 쉽고, 이러면 큰 차이를 만들기 어렵다.
➔ 랜덤 포레스트는 서로 다른 피쳐 세트를 임의로 구성해 개별 모델을 학습

랜덤 포레스트 Random Forest

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

parameters = {'max_depth': [3, 10, None]}

random_forest = RandomForestClassifier(n_estimators=100,
                                      criterion='gini', min_samples_split=30, n_jobs=-1)

grid_search = GridSearchCV(random_forest, parameters,
                           n_jobs=-1, cv=3, scoring='roc_auc')

grid_search.fit(onehot_train_X, train_y)
print(grid_search.best_params_)
```

```
{'max_depth': None}
```

```
random_forest_best = grid_search.best_estimator_
pos_prob = random_forest_best.predict_proba(onehot_test_X)[: , 1]
print('The ROC AUC on testing set is:{0:.3f}'
      .format(roc_auc_score(test_y, pos_prob)))
```

The ROC AUC on testing set is:0.707

성능이 약간 더 향상되는 것을 볼 수 있다.

부스팅 Boosting

- 학습 모델을 순차적으로 연결하여 최종 결과를 내는 방법
(Addictive Learning이라고도 불림)
- 한 모델에서 먼저 결과를 내고,
이 결과를 입력으로 다음 모델이 이어서 결과를 내는 방식
- 예) 이미지에서 얼굴 영역 찾기
 - 얼굴이 아닌 영역이 보통 더 넓기 때문에 찾기 쉽다.
 - 얼굴이 아닌 영역을 가리는 모델을 학습 시키고 순차적으로 실행
 - 한번 실행할 수록 얼굴이 아닌 영역을 제거하여 점점 입력 이미지가 작아지고 이는 피쳐 공간의 축소로 이어진다.

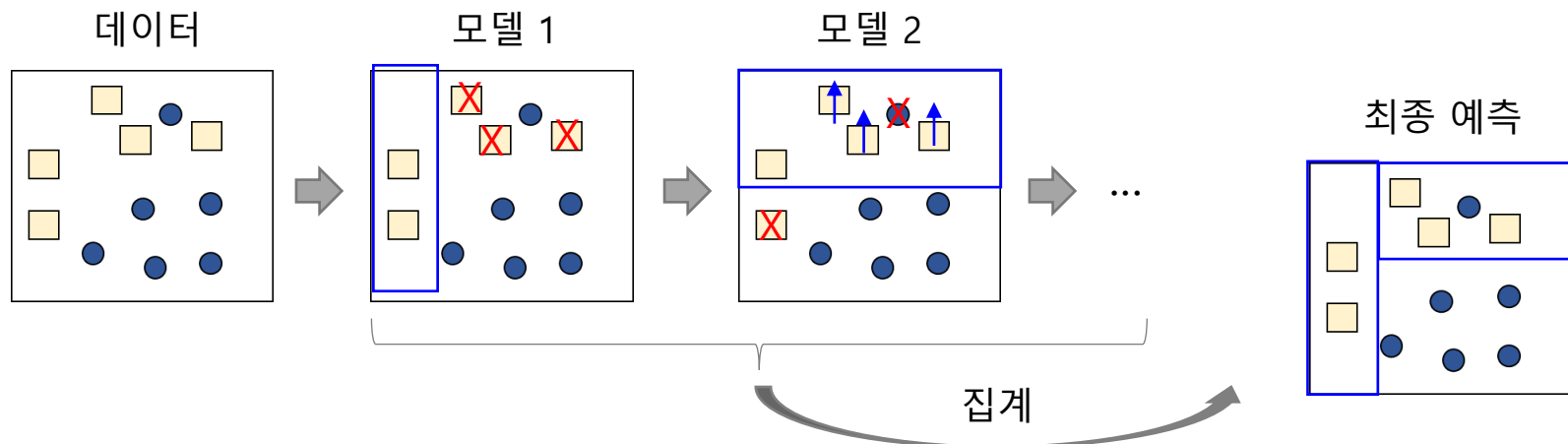
의사 결정 트리 기반 부스팅 기법

- AdaBoost (Adaptive Boosting)
- XGBoost (Extreme Gradient Boosting)
- LightGBM (Light Gradient Boosting Method)

많은 머신러닝 문제에서 XGBoost나 LightGBM이
딥러닝 기반 모델보다 더 좋은 성능을 보이고 있음
(예: Kaggle 우승 알고리즘들)

AdaBoost

- 정확도가 높지 않은 약한 학습 모델들을 연속적으로 이어 붙여서 결과를 점진적으로 개선하는 방법
 - 한 기계학습 모델의 예측 결과를 다음 모델 학습에 반영
 - 앞에서 잘 못 맞춘 데이터 샘플에 더 가중치를 주어서 다음 모델에서 더 집중적으로 배울 수 있도록 함



AdaBoost in Python

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score

decision_tree = DecisionTreeClassifier(criterion='gini',
                                       min_samples_split=30, max_depth= 3)

clf = AdaBoostClassifier(base_estimator =decision_tree,
                        n_estimators =50, learning_rate=1)

clf.fit(onehot_train_X, train_y)

pos_prob = clf.predict_proba(onehot_test_X)[:, 1]

print('The ROC AUC on testing set is: {0:.3f}'
      .format(roc_auc_score(test_y, pos_prob)))
```

The ROC AUC on testing set is: 0.704

AdaBoost in Python

- AdaBoostClassifier 객체 생성 파라미터
 - base_estimator : Boost에 사용할 학습 모델
 - n_estimators : 모델을 최대 몇 개까지 이어 붙일 것인지
 - learning_rate : 뒤에 붙는 모델일 수록 전체 결과에
작은 영향을 주도록 하는 변수
(1이면 모든 모델 동일)

Targeted Marketing

타겟 마케팅 Target Marketing

- 온라인 광고는 수십억 달러 규모의 사업
- 광고주 또는 광고 에이전시는 잠재적인 고객에게 광고 메시지가 잘 전달되도록 다양한 채널(인터넷, 모바일 앱 등)을 사용한다.
- 광고의 효과를 더 보기 위해서 많은 기업들이 특정 사용자 그룹을 대상으로 광고를 제작하고 전달하는 **타겟 마케팅**에 관심을 두고 있다.
 - 예) 20대 대학생들을 대상으로 000 광고
 - 예) 스포츠에 관심이 많은 사람들을 대상으로 하는 000 광고

타겟 마케팅 Target Marketing

- 타겟 마케팅을 위해서는 아래 사항들을 고려해야 한다.
 - 무엇을 광고할 것인가?
 - 누구에게 광고할 것인가? (타겟 그룹)
 - 어떻게 광고할 것인가? (콘텐츠/메시지)
 - 어디서 광고할 것인가? (광고 채널)
- 여러 가능한 조합들이 존재하기 때문에 **타겟 광고의 효과**를 미리 예측해 보고 가장 좋은 조합을 선택하여 타겟 마케팅을 수행한다.
 - ➔ 온라인 광고의 효과는 광고 클릭스루 비율로 평가해 볼 수 있다.

광고 클릭스루 비율

AD Click-Through Rate (CTR)

- 광고의 효용성을 평가하기 위한 보편적인 측정 기준 중 하나
- 전체 페이지 뷰 횟수 대비 해당 광고를 클릭해 본 비율
- CTR이 높을 수록 광고가 잘 타겟팅 됐음을 뜻하며, 온라인 광고가 성공적이라고 추측할 수 있는 근거가 된다.

머신러닝을 통해 광고 클릭스루 비율을 예측할 수 있다면

타겟 광고가 성공을 거둘 수 있을지 미리 가늠해 볼 수 있을 것이다.

광고 클릭스루 데이터 예

광고 콘텐츠, 채널, 사용자에 대한 피처와 광고 클릭 여부가 포함됨

Ad category	Site category	Site domain	User age	User gender	User occupation	Interested in sports	Interested in tech	Click
Auto	News	cnn.com	25-34	M	Professional	True	True	1
Fashion	News	bbc.com	35-54	F	Professional	False	False	0
Auto	Edu	onlinestudy.com	17-24	F	Student	True	True	0
Food	Entertainment	movie.com	25-34	M	Clerk	True	False	1
Fashion	Sports	football.com	55+	M	Retired	True	False	0
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Food	News	abc.com	17-24	M	Student	True	True	?
Auto	Entertainment	movie.com	35-54	F	Professional	True	False	?

CTR Prediction Dataset

- Kaggle에서 제공하는 Click-Through Rate Prediction Dataset에 Decision Tree 알고리즘을 적용해 보자.
 - <https://www.kaggle.com/c/avazu-ctr-prediction>
- 샘플 데이터 다운로드
 - 전체 데이터 중 20,000개 데이터만 사용하도록 하자.
 - 수업 자료실에서 다운로드
'ctr_data_20000.csv'

CTR Prediction Dataset

변수 명	설명
id	광고 ID
click	클릭 여부 (클릭 : 1, 클릭 X : 0)
hour	페이지 뷰 시점 YYYYMMDDHH
banner_pos	배너 위치 (0 혹은 1)
site_id	웹사이트 ID
site_domain	웹사이트 도메인
site_category	웹사이트 카테고리
app_id	모바일 앱 ID
app_domain	모바일 앱 도메인
app_category	모바일 앱 카테고리
device_id	모바일 기기 ID
device_ip	모바일 기기 IP
device_model	기기 모델
device_type	기기 종류 (Tablet, Smartphone, ...)
device_conn_type	네트워크 접속 종류 (WiFi, LTE, ...)
C1, C14-C21	그외 익명화 된 변수

데이터 읽어오기

데이터를 읽어와서 컬럼 확인

```
import pandas as pd

data = pd.read_csv('ctr/ctr_data_20000.csv')

data.columns
```

```
Index(['id', 'click', 'hour', 'C1', 'banner_pos', 'site_id', 'site_domain',
       'site_category', 'app_id', 'app_domain', 'app_category', 'device_id',
       'device_ip', 'device_model', 'device_type', 'device_conn_type', 'C14',
       'C15', 'C16', 'C17', 'C18', 'C19', 'C20', 'C21'],
      dtype='object')
```

쓸모 없는 컬럼들 삭제

id, hour, device_id, device_ip

```
data = data.drop(['id', 'hour', 'device_id', 'device_ip'], axis = 1)
```

범주형 데이터 처리 Categorical Data

- 머신러닝 라이브러리들이 수치형 데이터를 기반으로 돌아가기 때문에 범주형 데이터를 수치화 해야 한다.
 - 예) 성별, 연령대 (10대, 20대, 30대, ...), 스포츠 관심 여부 등
- 어떻게 범주값을 수치값을 할당 할 수 있을까?
 - [잘못된 생각] 임의의 숫자를 배정한다.

관심사 (피처)
기술
기술
과학
인문
예체능



범주값	수치값
기술	1
과학	2
인문	3
사회	4
예체능	5



관심사 (피처)
1
1
2
3
5

관심사 피처는 원래는
수학적 의미가 전혀 없음에도 불구하고
숫자가 할당이 되면서
이상한 수학적 의미가 함께 포함되어 버린다.
(예: 예체능은 기술보다 큰 값?)

원 핫 인코딩 벡터 One-Hot Encoding Vector

- 각 범주를 벡터의 각 성분에 매핑하되, 해당 범주의 속성 값만 1을 갖게 하고 나머지 성분은 0을 같도록 하는 방법
 - 피처를 범주 수 만큼 늘린다고 생각

5개 범주를 가진 피처

관심사 (피처)
기술
기술
과학
인문
예체능



5개의 이진 피처

관심사_기술	관심사_과학	관심사_인문	관심사_사회	관심사_예체 능
1	0	0	0	0
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	0	1

범주형 데이터 처리

- DictVectorizer 클래스를 이용하면 원 핫 인코딩을 손쉽게 할 수 있다.
 - 19개 피처가 3244개 피처로 늘어 났다.

```
from sklearn.feature_extraction import DictVectorizer

data_X = data.loc[ :, data.columns != 'click' ].astype('str')
data_y = data.loc[ :, data.columns == 'click' ].astype('int').values.ravel()

train_X, test_X = data_X[0:15000], data_X[15000:]
train_y, test_y = data_y[0:15000], data_y[15000:]

dict_one_hot_encoder = DictVectorizer(sparse=False)
onehot_train_X = dict_one_hot_encoder.fit_transform(train_X.to_dict('records'))
onehot_test_X = dict_one_hot_encoder.transform(test_X.to_dict('records'))

print(train_X.shape)
print(onehot_train_X.shape)
```

```
(15000, 19)
(15000, 3244)
```

Decision Tree 분류기 학습

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

parameters = {'max_depth': [3, 10, None]}
decision_tree = DecisionTreeClassifier(criterion='gini',
                                       min_samples_split=30)

grid_search = GridSearchCV(decision_tree, parameters,
                           n_jobs=-1, cv=3, scoring='roc_auc')

grid_search.fit(onehot_train_X, train_y)
print(grid_search.best_params_)

{'max_depth': 10}
```


성능 평가

```
decision_tree_best = grid_search.best_estimator_  
pos_prob = decision_tree_best.predict_proba(onehot_test_X)[: , 1]  
  
from sklearn.metrics import roc_auc_score  
print('The ROC AUC on testing set is: {0:.3f}'  
      .format(roc_auc_score(test_y, pos_prob)))
```

The ROC AUC on testing set is: 0.688

Lab 3-2: Tree-based Algorithms

Lab

- NH Data Science Course

- All code examples are written in Python 3
- Please do not hesitate to ask questions!
- TAs
 - 김규태
 - 최예림
- 모든 Lab 자료는 eX-campus에 업로드 되어 있습니다.